FPGA-Based Real-Time Anomaly Detection for Industrial Robotic Arms

Kevin Zheng, A/Prof. Mehrtash Harandi

***Abstract*—Industrial robots are increasingly used in safety-critical applications where undetected anomalies can cause equipment damage and safety incidents. Existing anomaly detection solutions often rely on software based neural networks running on CPUs and GPUs which have inference latencies in the order of milliseconds or high power consumption, making them unsuitable for time critical edge applications. This paper presents a field programmable gate array (FPGA) implementation of a convolutional neural network (CNN) based anomaly detection system, achieving an inference latency of 120 nanoseconds at 100 MHz, an improvement of nearly 6 orders of magnitude over the microcontroller implementation of Kayan et al. running on a 64 MHz 32-bit Arm Cortex-M4 CPU. The system is estimated to consume 1.684 W of power during inference and maintains all outputs within a mean absolute error of 0.37% and maximum absolute error of 3.81% of each feature's standard deviation, achieving an F1 score of 0.951, within 0.002 of the software model. These results demonstrate that FPGA based inference can enable real-time anomaly detection for safety critical systems where software-based approaches are limited by power or latency.**

***Index Terms*—Anomaly Detection, FPGA, Real-Time, Robotic Arms**

I. Introduction

A. Background & Context

INDUSTRIAL robots are becoming increasingly deployed in manufacturing environments, with annual installations in 2024 exceeding 500,000 units globally [1]. This rise has been driven by benefits such as increased productivity and precision, as well as growing integration with network-based control systems which enable real-time monitoring, remote operation, and synchronisation between multiple robots [2]. However, despite their robustness, industrial robots remain susceptible to mechanical faults caused by repetitive motion, overheating, programming errors, and environmental damage [3], [4]. These faults manifest as subtle changes in the robot's motion known as anomalies, and if left undetected, could result in equipment damage and serious safety incidents. Therefore, the window between anomaly onset and detection directly determines both system safety and operational downtime. Fast and reliable anomaly detection is therefore a critical requirement in industrial settings.

Beyond mechanical degradation, a more difficult to detect source of anomalies arises from the network-based communication protocols used to connect industrial robots for remote monitoring and synchronisation [2]. One such protocol, Real-Time Data Exchange (RTDE), has been shown to allow unauthorised reading of the robot's state and unauthorised writing to registers and outputs [6]. This allows an adversary to spoof joint waypoints and sensor data being sent to the remote controller, causing the robot to execute unpredictable motion. Since this class of anomalies compromises the data used by the conventional detection systems, detecting it requires a monitoring mechanism which does not depend on the integrity of the robot's reported state.

Moreover, in recent years, anomaly detection research for industrial systems has gradually moved towards running inference closer to the physical sensor to reduce latency, with earlier systems relying on cloud-based inference [5]. This reflects the shift in embedded machine learning toward on-device inference due to low latency being of high importance to safety critical systems. Therefore, for the purposes of our system, the detection should be done on the edge rather than the cloud to minimise the round-trip latency. Moreover, since industrial systems are expected to run continuously for extended periods of time, the system should operate at a low power to avoid requiring frequent maintenance.

Kayan et al. [5], [7] address both these issues by using an externally mounted IMU sensor, which is decoupled from the robot's controller, to detect movement-based anomalies independent of network integrity. This sensor being mounted on the robotic arm itself ensures that they do not need to pay for the latency of sending the data to the cloud for processing and can perform the entire pipeline on the edge. Their system achieves a high detection accuracy using lightweight 1D-CNN and LSTM models running on a microcontroller edge device. However, this comes at the cost of the inference latency, with their system having a latency of 115–120 milliseconds [7], meaning their practical detection frequency achievable is 8 Hz. Since this inference time exceeds the 100 ms sampling interval of their sensor, the system is compute-bound and cannot keep pace with its own sensor. For a robot performing fine, high-speed manipulation [8], several hundred milliseconds may be enough for an anomalous trajectory to cause a collision or damage before it is flagged. Existing low latency alternatives, relying on using GPUs or Jetsons, achieve sub-millisecond inference at the cost of power, with these using tens to hundreds of watts making them impractical for continuous edge deployment. While there exist NPU (Neural Processing Unit) and TPU (Tensor Processing Unit) co-processors designed for AI (Artificial Intelligence) inference at the edge [9], discrete co-processors such as Google's Coral Edge TPU require a separate host device. This host device results in the overall system having a larger power draw despite the relatively low power of the co-processor. Moreover, since these co-processors cannot directly take in sensor data, they rely on the host to transmit the data via PCIe or USB which

introduces extra communication overhead which bottlenecks their achievable latency to the microsecond range [10] despite the co-processors themselves being capable of many tera-operations per second (TOPS). It has been observed that the overhead introduced by using a co-processor may cause it to perform worse than a regular CPU for very small models [10]. While integrated NPUs, such as Arm's Ethos-U, avoid the external bus, their inference remains software-mediated and requires a runtime driver to submit a pre-compiled command stream to the NPU [11] which may bottleneck their compute latency for small models despite its high TOPS [9].

The host device may also introduce non-deterministic jitter and context switching overheads due to the operating system. A common theme across these architectures is the presence of a software layer which sits between the sensing and detection, whether this be an operating system, driver, or firmware managing a peripheral call. Each layer adds additional latency and uncertainty which limits the latency the design can achieve. In contrast, using a dedicated hardware circuit can eliminate this latency entirely, as the entire process from inference to detection can be performed entirely in hardware.

To the best of our knowledge, no FPGA implementation has been applied to external motion monitoring of robotic arms, which [7] shows can detect anomalies independent of the robot's reported state. In this paper, we address this gap by proposing an FPGA-based anomaly detection system that performs the inference and detection pipeline directly in reconfigurable hardware, eliminating the software overhead from the critical path. We show that this approach achieves an inference latency of 120 nanoseconds, while estimated to consume 1.684 W of power during inference and 0.227 W while idle, comfortably within the range required for continuous edge operation. This removes inference as a contributor to detection latency, leaving it bounded by the sensor rather than by computation, at a power draw compatible with continuous deployment. This is a combination which neither the CPU, GPU, nor co-processor implementations provide. The core principle of eliminating the software overhead from the inference pipeline entirely is not unique to anomaly detection and can be extended to other domains where latency is critical such as in experimental physics, high frequency trading, real-time control systems, and real-time computer vision in autonomous vehicles.

B. Literature Review

Existing anomaly detection systems are shown in Table I.

From Table I, we see that most of the existing anomaly detection solutions mainly rely on GPUs to accelerate inference to the required sub-millisecond level. These GPUs run very large models, making these solutions unsuitable for edge deployment due to their size and computation complexity as well as the relatively large power consumption of the GPU itself.

A prominent theme throughout many entries in the table is the tradeoff between latency and power. We see that the GPU solutions have much lower latency, yet consume much more power [5], [14], [15], [17], [18], compared to the edge device solutions [7], [12], [13], [16] which have much lower power at the cost of greater latency. Therefore, in applications which require both low inference latency and low power, these would not be suitable. Using FPGA based solutions allows custom hardware circuits to be built which can minimise inference latency and lower the power of the design as we can design a hardware circuit tailored towards the specific approach. We see this as a general trend in the FPGA based approaches in the table, where they achieve low latency with relatively low powers.

TABLE I
EXISTING ANOMALY DETECTION METHODS

Ref.	Model	Hardware	Latency	Power	Application
[5]	CNN	Nvidia RTX A6000	36.97 µs	300 W	Robotic arms
[5]	LSTM	Nvidia RTX A6000	51.80 µs	300 W	Robotic arms
[7]	CNN	Nicla Sense ME	115 ms	66 mW	Robotic arms
[7]	LSTM	Nicla Sense ME	176 ms	66 mW	Robotic arms
[12]	IF	ESP32 WROVER	16 ms	0.19–0.7 W*	Submersible
[13]	CNN	Raspberry Pi 4	12.3 ms	2.7–6.4 W*	IoT network
[14]	AE	Nvidia RTX 4070 Ti	<1 ms	285 W	Latent spaces
[15]	AE	Nvidia RTX 4060	125 ms	115 W	Industrial robots
[16]	ViT/MP	Nvidia Jetson AGX	53 ms	1.2–15 W*	UAV anomalies
[17]	CNN	Nvidia RTX 3090	2.9 ms	15–350 W*	Transformers
[18]	VLA	Nvidia GeForce RTX	31.2 ms	56–350 W*	Robotic arms
[19]	MLP	Xilinx XC7A200T	20 ns	0.13–2.4 W*	Motor faults
[20]	CNN	Xilinx PYNQ-Z1	0.42 ms	1–2 W*	Bearing faults
[21]	RF	Xilinx VU9P	30 ns	78 W*	Higgs decay
Ours	CNN	Xilinx AUP-ZU3	120 ns	0.23–1.7 W	Robotic arms

*The paper did not provide the value, so it was estimated from datasheets (idle to typical load).

It is worth noting that the latency and power figures in Table I are not directly comparable since they arise from models of differing complexity and are evaluated on different tasks and datasets. The motor fault detector, for example, achieves 20 nanoseconds of latency using a MLP based approach, which is substantially simpler than the AE, CNN, and LSTM architectures in [5], [7], [13]–[15], [17], [20] which process raw time series data or image data. Similarly, sub-millisecond GPU results in [5] and [14] come from models which are orders of magnitude larger than the microcontroller deployed networks in [7] and [12], which is a deliberate design tradeoff rather than a hardware limitation alone. This distinction matters since it means the latency and power tradeoff is confounded with model choice. A fair comparison would require us to keep the model architecture constant while varying the hardware platform, which the anomaly detection systems in Table I do not do. As a result, we aim to address this by directly accelerating the CNN architecture used in [7] which was selected as the case study since its model, code, datasets, and microcontroller baseline are all publicly available, allowing the hardware platform to be isolated as the only variable.

While these FPGA approaches for anomaly detection already exist, they are currently tailored towards monitoring the electrical and mechanical health of machines using internal signals. As a result, they do not address the cybersecurity issue presented by Kayan et al. in [7] as a machine could be in perfect health but behave anomalously due to the waypoints of its joints being spoofed, something these current methods cannot detect.

Therefore, the literature exposes two major gaps in the current field.

Firstly, existing anomaly detection systems for robotic arms achieve either low-latency or low power, not both. The GPU based approaches in [5] achieve sub-millisecond inference at 300 W, while [7] draws 66 mW but requires over a hundred milliseconds to complete inference. Co-processors reduce power but introduce host and communication overhead which limits their latency for small models typically deployed on the edge.

Secondly, the existing FPGA implementations focus on health monitoring using internal signals, and no FPGA implementation has been applied to externally sensed motion monitoring.

C. Problem Definition

In this paper, we present an FPGA based solution which addresses these gaps. We investigate the minimum inference latency achievable by an FPGA implementation of the CNN in [7] when all the software layers are removed from the critical path, while remaining within an edge compatible power budget of 2 W and maintaining the accuracy of the software model. Although the robotic arm application does not itself require nanosecond scale inference, minimising latency demonstrates the performance ceiling of this approach for domains where latency is a critical constraint such as in high frequency trading. Through this, we seek to show FPGA as a viable solution for inference on the edge in real-time systems which require both low inference latency and low power, a combination current implementations do not provide. Specifically, this paper addresses the following research question: *can the CNN-based anomaly detector of [7] be implemented on an FPGA with sub-microsecond inference latency, within a 2 W power budget, and without measurable loss of detection performance?*

II. System Overview

A. System Architecture

The system presented in [7] consists of an IMU sensor connected to a Nicla Sense ME microcontroller unit.

Kayan et al. trained a CNN on 26 hours of IMU data sampled at 10 Hz from a robotic arm following a repetitive motion. This network predicts the future IMU state of the arm using its current IMU state. The prediction is compared to the true sensor value from which RMSE is calculated. The RMSE values across a window of time are then summed together, and if the total error exceeds a predefined threshold, the window is classified as anomalous. This is because a very high RMSE would imply the arm is not following its predefined trajectory and acting anomalously.

The microcontroller runs the pipeline from the inference of the CNN to the computation of the RMSE during the window and finally a decision of whether an anomaly has been detected. Due to the 115–120 ms inference latency of the microcontroller, the highest sampling rate their system could operate at was bound at around 8.7 Hz.

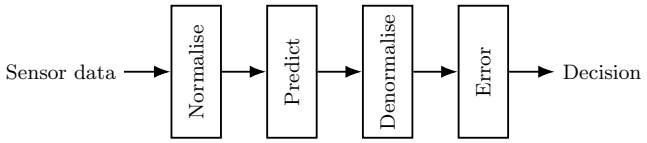


Fig. 1. System block diagram.

Although the present application may not require extremely low latency inference, this study aims to minimise the FPGA inference latency to demonstrate its potential as a viable alternative to existing GPU and TPU based solutions in domains where inference latency is a critical constraint, such as those previously mentioned.

There are multiple questions to answer in order to implement such a system on an FPGA. Firstly, one must determine precision of computation on an FPGA. Secondly, CNNs typically require many tens of thousands of multiply accumulate (MAC) operations per inference pass, so one must determine how to maintain minimal inference latency while ensuring the design's resource usage does not exceed the board's physical limits.

Kayan et al. [7] found that full integer quantisation (INT8) caused the model's accuracy to drastically drop from 96% to 60%, so floating point was retained. They also found that quantising from float32 to float16 resulted in a negligible drop in performance. However, since their hardware did not support the data type, they were forced to use float32 which required double the space for a negligible gain in performance. Since using float16 would halve the amount of resources without impacting performance, we choose this precision for our implementation. While we could have considered fixed point, we wanted to remain faithful to the original study in [7] to ensure the only variable was the hardware model so results remain directly comparable.

The resource challenge was the most difficult to solve in our design. Since the fundamental operation in all layers is the dot product, a resource sharing approach was adopted to minimise the number of resources required for the design. The specific details will be discussed in Section III.

A high-level overview of the system presented in [7] is shown in Fig. 1.

The latency measured in this study is the number of cycles between the predict block receiving the window's last normalised input, and it outputting a valid result.

The normalisation and denormalisation blocks are responsible for shifting and scaling the data by the training mean (μ_{training}) and standard deviation (σ_{training}). This was required since the CNN in [7] takes in normalised inputs. Since division is expensive to implement in hardware, we pre-computed the reciprocal of σ_{training} and used a multiply instead of a divide operation.

The predict block represents the inference stage where the normalised data is put into the CNN to obtain a prediction of the future state of the arm's motion.

The error block computes the accumulated root mean square error (RMSE) between the predicted state and the true state of the arm and determines whether an anomaly has been detected.

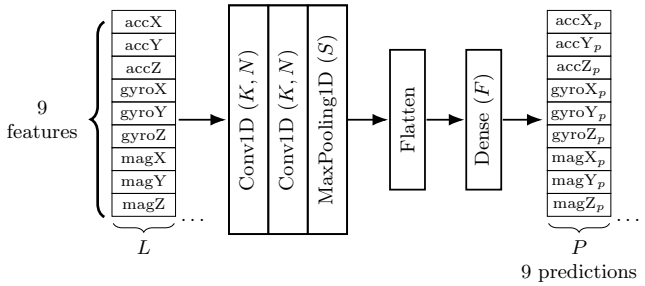


Fig. 2. CNN block diagram, redrawn from [7].

TABLE II
SOFTWARE CNN PARAMETERS

Layer	Input Size	Output Size	Filter Size	Num. Filters
Conv 1	45×9	43×8	3×9	8
Conv 2	43×8	41×8	3×8	8
Max Pooling	41×8	13×8	3×1	–
Flatten	13×8	1×104	–	–
Dense	1×104	1×9	104×9	–

The hardware model developed contains the same blocks as the system above.

B. CNN Architecture

1) Software Architecture: The layout for the CNN described in [7] is shown in Fig. 2. It consists of two convolutional layers with ReLU activation functions, a max pooling layer, and finally a dense layer. The data type used was float32.

The model takes in a window of 45 consecutive IMU samples, with each containing 9 features, giving a 45×9 input. Each convolutional layer is 1D convolution along the time axis.

The parameters of the model presented in [7] are given in Table II.

Note that not all the 41×8 inputs to the max pooling layer are used. Since 41 is not divisible by 3, only 39 of the 41 values per channel can be grouped together into complete windows of 3, so the final two values for each channel are discarded.

2) Hardware Architecture: The software CNN uses a batching approach, where it waits for the entire input image to arrive before beginning computation. This is impractical on an FPGA as processing the entire image all at once with minimal latency would require far more hardware than the board provides. The design therefore uses a streaming approach, where parts of the window are processed as they arrive. This allows hardware to be re-used across clock cycles, substantially reducing resource usage.

Data is stored in contiguous arrays, which keeps it in a flattened form throughout, so no reshaping is required and the flattening layer could be removed from the design.

IMUs transmit data serially, so for simplicity, the module is assumed to receive one float per clock cycle. The first convolutional layer (CL1) therefore has an input size of 1×1. Each layer's output depends on the number of filters or output channels. CL1 applies eight 3×9 filters to its inputs, producing a 1×8 output. CL2 takes in this input and applies its eight 3×8 filters to also produce a 1×8 output. The max pooling layer then takes the maximum over a window of three consecutive samples for each of the 8 channels in parallel and produces a 1×8 output. Finally, the dense layer uses these values to produce a prediction for the 9 output classes, giving a 1×9 output.

TABLE III
HARDWARE CNN PARAMETERS

Layer	Input Size	Output Size	Filter Size	Num. Filters
Conv 1	1×1	1×8	3×9	8
Conv 2	1×8	1×8	3×8	8
Max Pooling	1×8	1×8	3×1	–
Dense	1×8	1×9	–	–

The final parameters of the hardware model are given in Table III.

Unlike Table II which gives the dimensions of the entire input window, Table III gives the data each layer receives and produces per operation in the streaming design.

C. RMSE Architecture

1) Why RMSE Rather Than MSE: While in theory we could have used the mean square error (MSE) rather than RMSE since the square root function is monotonic, there were two reasons why we decided to implement our system with an RMSE.

Firstly, the original study in [7] used RMSE and we wanted to reproduce their results. Secondly, since the thresholds of the anomalies, as shown in Table VII (see Section IV-C), were relatively large, not using the square root would result in RMSE of many valid windows being unable to be represented in float16 due to overflow. Therefore, the square root was used to limit the range of values and ensure the float16 data type could be used for the entire pipeline, saving resources.

Most parts of the RMSE computation in the hardware implementation were straightforward, with these being the subtraction, squaring, and summing operations. The square root computation and running sum, however, required some additional consideration.

2) Square Root: For the square root, a single cycle look up table approach was chosen to minimise the latency. Other methods, such as Newton's method, may require fewer resources but take multiple cycles. Since one of the objectives is to minimise inference latency, we went with the single cycle approach.

The simplest implementation would be to use the float16 value as a lookup key. However, this is expensive, since it has a space complexity of $O(W2^W)$, where W is the width of the key, therefore requiring 128 KB of memory and 32 block RAMs (BRAM) which store 36 Kb of data each.

We can optimise this through two important observations. Firstly, the sum of squares is always non-negative, so we can drop the sign bit. Secondly, notice what happens when we square root a float16 value.

TABLE IV
RESOURCE USAGE AND $f_{\max}$ OF ARITHMETIC MODULES

Module	LUT	DSP	$f_{\max}$ (MHz)
Float Adder	211	0	172.86
Float Multiplier	37	1	293.08
Float Greater Than	21	0	846.02

For any float16 represented as $N = M \times 2^E$, where $M \in [1, 2)$ is the significand and E is the unbiased exponent, taking the square root of N gives

$$\sqrt{N} = \sqrt{M \times 2^E} = \sqrt{M}\, 2^{E/2}. \tag{1}$$

Considering the cases where E is either odd or even, we obtain

$$\sqrt{N} = \begin{cases} \sqrt{M}\, 2^{E/2}, & E \text{ even}, \\ \sqrt{2M}\, 2^{\lfloor E/2 \rfloor}, & E \text{ odd}. \end{cases} \tag{2}$$

In both cases, the new significand lies in the interval $[1, 2)$, so we can read it directly from a lookup table whose key is the parity of E combined with the 10 mantissa bits of N, which returns the mantissa of the result. This only requires $10 \times 2^{11} = 20$ Kb, which fits within one BRAM.

3) Rolling Sum: Since the running sum of the last W RMSE values, where W is the window size, needs to be computed, a sliding window approach was used to allow the new sum to be calculated in $O(1)$ time by adding the incoming RMSE value and subtracting the oldest value leaving the window. This avoids having to repeatedly sum up the entire window every time a new RMSE value arrives.

III. IMPLEMENTATION

A. Floating Point Arithmetic

To accurately replicate the software CNN, the IEEE 754 standard of adding and multiplying floating point numbers was followed when designing the hardware components, with these being the float adder, float multiplier, and float greater than modules.

1) Rounding: Although the IEEE 754 standard includes guard, round, and sticky bits for correct rounding, these were not used in the design to save resources. While this may decrease the accuracy of the system, our results indicate the effect is negligible.

2) Comparison: Due to the way in which IEEE 754 numbers are formatted, with the exponent bits being more significant than the mantissa bits, we can quickly check which of two floats are greater by treating it as a comparison between two integers and handling the infinity and NaN cases where appropriate.

Furthermore, since the ReLU activation functions are used, the max pooling layer will only receive values which are greater than or equal to zero so the sign bit can be ignored. Therefore, this operation is equivalent to a comparison between two 15-bit unsigned integers.

3) Resource Usage & Timing: The resource usage of the components is shown in Table IV.

The design targets the AUP-ZU3 development board which provides around 70K look-up tables (LUTs), the FPGA's basic logic elements, 360 digital signal processing (DSP) slices, which are dedicated multiplier blocks, and 216 BRAM blocks. From Table IV, we see that we would only be able to implement either 334 float adders, or 360 float multipliers without any other additional logic.

If we simply implemented everything in Table IV naively and assuming no additional logic, we would require 455 float adders, and 480 float multipliers. This is a total of 114K LUTs which far exceeds our board's resource limit.

Although the $f_{\max}$ of the float adder is 172 MHz in isolation, routing delays in the final design limited the clock frequency to 100 MHz. This prevents two float additions being chained in a single cycle, since this would require around 11.6 ns, exceeding the 10 ns clock period before routing delays are even considered. Note that while it is theoretically possible to complete an add and multiply in one cycle as this requires 9.2 ns, it was not practically possible due to the delays caused by routing congestion.

B. Accumulation

We considered different approaches for accumulating numbers and their impact on latency.

Summing N inputs can be done with an accumulator, which re-uses one adder and adds one input per cycle, or with an adder tree, which adds pairs of inputs in parallel and halves the number of values at each level. The accumulator needs only one adder but takes N cycles, while the tree needs $N-1$ adders but takes only $\lceil \log_2 N \rceil$ cycles.

Since only one float addition fits in one clock cycle, each level of the tree is pipelined. This also allows temporal parallelism since multiple levels can process data simultaneously which improves throughput.

Since the goal was to minimise latency, the tree was chosen despite its larger size.

C. Dot Product

The dot product is fundamental for all layers in the neural network. It was constructed by using a multiply layer consisting of N multipliers followed by an N input float adder tree.

This means the number of float adders in an N input dot product module is $N-1$, and the number of float multipliers is N. Therefore, the latency is

$$L_{DP}(N) = \lceil \log_2 N \rceil + 1. \tag{3}$$

Let

$$R_{DP}(N) = N R_{FM} + (N-1) R_{FA} \tag{4}$$

be the resource usage of an N input dot product in LUTs, where R_{FM} and R_{FA} are the LUTs of a float multiplier and float adder.

D. Convolutional Layer

1) Theory of Operation: Suppose we had a kernel with dimensions $K_H \times K_W = K_S$ performing convolution on an $H \times W$ image with no padding and a stride of 1.

The very first convolution, with the kernel in the top left corner, would require $K_H - 1$ rows of the image, and an additional K_W inputs to form the first valid window. As a result, we would need $(K_H - 1) \times W + K_W$ inputs to get our first valid convolution.

For the subsequent operations, we would shift the kernel across by one then compute the convolution. This shifting behaviour is performed by using a shift register in hardware to store the data as shown in Fig. 3.

We eventually run into invalid windows when the kernel wraps, during these cycles no valid output will be produced.

Although the convolutional layer has C_{out} different output channels, all these channels use the same input data so we only require one shift register for the entire layer.

On each valid window, we use dot product modules working in parallel to multiply the weights and values to compute the convolution. This gives a latency of $\lceil \log_2 K_S \rceil + 1$.

2) Naïve Implementation: The naïve approach would be to have a dot product for each channel which multiplies the kernel by the valid window. This would require $C_{out}(K_S - 1)$ float adders and $C_{out}K_S$ float multipliers, resulting in the first and second layers requiring 51.8K and 45.9K LUTs respectively which already exceeds the resource limit without additional logic. Therefore, this is not feasible for our design and we must optimise our approach.

3) Pre-Compute Optimisation: We made three important observations which allowed us to lower the resource usage of the convolutional layers.

Firstly, there are K_W cycles between groups of valid windows due to our window wrapping around the image. This implies the naïve approach is not required since we do not need to compute a valid window on every clock cycle, so we can save hardware by using a different approach.

Secondly, most of the hardware remains idle in the naïve approach, and we are not utilising the temporal parallelism which pipelining the adder tree provides to our advantage. Through temporal parallelism, we re-use the same hardware across successive clock cycles without having to wait for its current computation to finish, allowing us to use fewer resources.

Finally, since the next window uses data from the current window, we can begin pre-computing results for the next window before it has fully arrived by re-using hardware.

Rather than waiting for the next window to arrive before beginning processing, we re-use the same dot product hardware during the invalid windows to begin pre-computing dot products for the next valid window using the data from the current window. This is done by having data from the current window and changing the weights input on every cycle using some number of dot product modules.

As new data arrives on each cycle, we begin multiplying and accumulating them with their respective weights. This also saves hardware, since we re-use the same hardware to compute these results.

4) Datapath Latency Analysis: We can see from the above that we have two independent data paths, one for the bottom of the kernel using the existing data, and one for the top of the kernel using the incoming data.

Let $K_{bot} = K_W(K_H - 1)$ be the size of the data used for the next window which is available at the end of the current window.

Suppose we use N_{PC} dot product modules for the bottom data path. To minimise the latency we split the number of output channels equally among them, so the number of channels each computes is $C_{PC} = \lceil C_{out}/N_{PC} \rceil$. We need C_{PC} cycles to finish inputting weights into the pipelined hardware, and we are measuring the latency relative to the last piece of incoming data which takes K_W clock cycles to arrive, so

$$L_{bot} = \max(L_{DP}(K_{bot}) + C_{PC} - K_W,\ 0)\,. \tag{5}$$

The top path is only dependent on the input size since it immediately processes the incoming input. Therefore, the latency is $L_{top} = L_{DP}(C_{in})$. Since C_{in} is a positive constant for the layers, L_{top} is also a positive constant.

The latency of the convolutional layer becomes the max of the top and bottom with one additional cycle to merge the results:

$$\begin{aligned} L_{conv} &= \max\left(L_{top}, L_{bot}\right) + 1 \\ &= \max\left(L_{DP}(C_{in}),\ L_{DP}(K_{bot}) + C_{PC} - K_W\right) + 1. \end{aligned} \tag{6}$$

Since L_{top} is fixed, to minimise the latency we would require any N_{PC} which satisfies

$$L_{bot} \le L_{top} \iff L_{DP}(K_{bot}) + C_{PC} - K_W \le L_{DP}(C_{in}). \tag{7}$$

Solving the above for the value of N_{PC} we obtain the same solution for both the convolutional layers, with this being $N_{PC} \ge 2$. Since L_{top} is limiting, there is no benefit to finishing precomputation earlier as using more dot product hardware to do so would consume more resources for no benefit. Therefore, we select the minimum value of $N_{PC} = 2$.

5) Optimisation Results: Using the above results, we obtain

$$L_{conv} = L_{DP}(C_{in}) + 1. \tag{8}$$

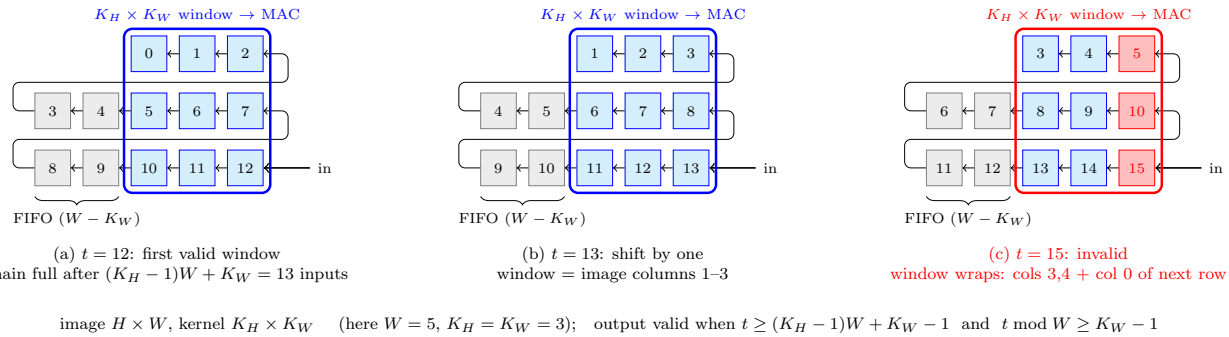
The resource usage would be approximately equal to the number of float adders and multipliers used for computing the top and bottom of the convolution and those needed for joining the results:

$$R_{conv} = N_{PC}R_{DP}(K_{bot}) + C_{out}\left(R_{DP}(C_{in}) + R_{FA}\right). \tag{9}$$

From this, we obtain the resource usages of the two convolutional layers as 10.5K and 23.4K LUTs, reducing the resource usage by 65%.

E. Max Pooling Layer

Since subsequent pooling windows overlap, the data needed for the next window is data from the previous window shifted across by one position. The oldest value is discarded, and a single new value is shifted in. As a result, only one new input



(a) $t = 12$: first valid window chain full after $(K_H - 1)W + K_W = 13$ inputs

(b) $t = 13$: shift by one window = image columns 1–3

(c) $t = 15$: invalid window wraps: cols 3,4 + col 0 of next row

image $H \times W$, kernel $K_H \times K_W$ (here $W = 5$, $K_H = K_W = 3$); output valid when $t \geq (K_H - 1)W + K_W - 1$ and $t \bmod W \geq K_W - 1$

Fig. 3. Convolution shift register example.

has to be read per window rather than having to re-compute the entire window. As a result, the max pooling layer uses a shift register to store its data.

Float greater than comparators are placed along the register, each taking a group of three adjacent values and outputting their maximum. To move to the next window, the contents of the shift register simply shift by one position. Since the comparators are connected at fixed positions along the shift register, the maxima values for the new window are automatically computed.

To get the maxima of three values we require two float greater than modules. From Table IV, we see using two in sequence would comfortably pass timing at 100 MHz, so this can be done with zero cycles of latency.

F. Dense Layer

1) Naïve Implementation: With every valid input the max pooling layer shifts all the positions by one which results in 104 new values to use for compute. The naïve approach would be to use a 104-input dot product for each of the 9 output channels. This would require $R_{DP}(936) = 231.9$K LUTs which is more than triple our available resource limit.

2) Resource-Sharing Optimisation: To avoid exceeding the resource limit, we used a resource sharing approach which gives different layers access at different points in time, similar to TDMA in telecommunication networks.

The dense layer gains access to the dot product hardware used for convolution, with these being the two 18 input dot product modules for CL1, and the two 16 input and the eight 8 input dot product modules for CL2. The collections of these dot product modules will be referred to as "banks".

We add two extra eight input dot product modules to ensure we can meet the schedule of the system (Fig. 4).

3) Scheduling: The waveforms for the dense layer are shown in Fig. 4. It is important to note that the waveforms are periodic with a period of $K_{W,1} = 9$. The notation $M \times N$ means we have a dot product bank with M N-input dot product modules. There are 9 output channels labelled $D_0, D_1, \ldots, D_8$.

The arbiter must give the convolutional layers the highest priority to avoid the pipeline being stalled. A low busy signal shows the hardware is free for the dense layer to use.

TABLE V
RESOURCE USAGE OF FINAL DESIGN

Resource	Utilisation	Available	Utilisation (%)
LUT	61663	70560	87.39
FF	16119	141120	11.42
BRAM	7.50	216	3.47
DSP	163	360	45.28

During each period, we place 32 values from each channel into the 2×18 and 2×16 banks, and 8 values into the 1×8 bank. On each cycle, we place 8 values from each channel into the 9×8 bank. Overall, we compute 104 values for each channel per 9 cycles, meeting the compute requirement.

To ensure minimal latency, we compute the result one period in advance to ensure they can be summed with the results of the final sample immediately when it is available rather than having to wait for all the results to finish computing.

4) Optimisation Results: The latency of the dense layer is therefore the latency of the dot product calculation with one addition cycle to join the results, so

$$L_{dense} = L_{DP}(D_{in}) + 1. \tag{10}$$

Since the dense layer only needs two extra 8 input dot product modules, the resource usage is around $2R_{DP}(D_{in}) + D_{out}R_{FA} = 5.45$K LUTs, far below the naïve 231.9K LUTs.

G. Final Results

1) Latency: The latency of the CNN is $L_{conv,1} + L_{conv,2} + L_{dense} = 12$ cycles. Since we are running at 100 MHz, this is equivalent to 120 ns. The latency from the last raw input of a window to a decision on whether the window is anomalous is 80 ns. The power consumption was estimated using Vivado's post-implementation power analysis, giving 1.684 W during inference and 0.227 W while idle. These are estimates rather than on-board measurements.

2) Resource Usage: The resource usage of the final design, including the test vector and the Xilinx integrated logic analyser (ILA), an on-chip debug core which records internal signals, is shown in Table V, where FF denotes flip-flops (registers).

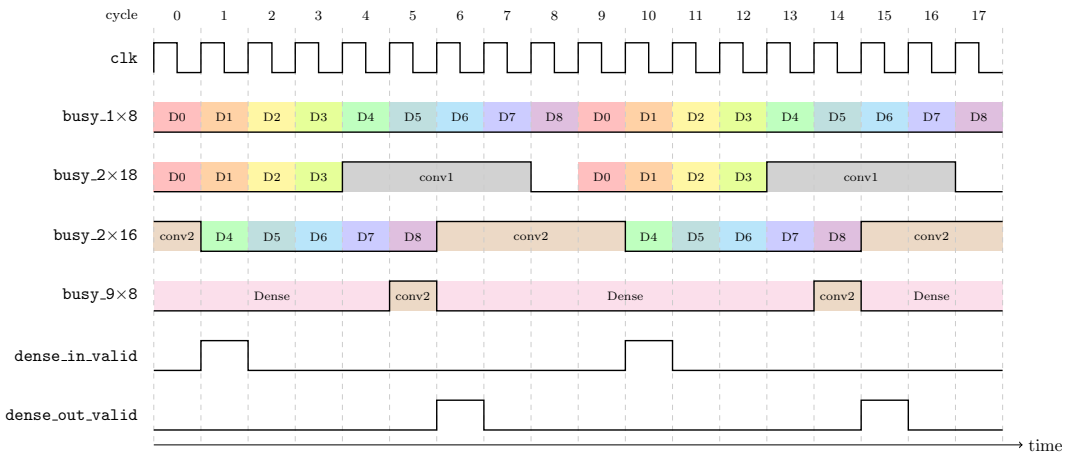


Fig. 4. Scheduling of dot product hardware.

IV. Results & Discussion

A. Verification and Inference

The design was run on the AUP-ZU3 board to verify the latency. The outputs were captured using the ILA. A 600-row test vector from the hitting arm dataset was streamed into the design from on-chip memory, and the accumulated RMSE values captured on the board matched the simulation bit for bit, with both reporting the same 282 anomalous windows out of 355.

The 120 ns latency is approximately one million times lower than the inference latency of the microcontroller implementation in [7]. Although latency comparisons should be treated with caution since different models will have varying computational complexities, this comparison is much more meaningful since both hardware platforms are running the same model. This result therefore indicates that the latency of a model can be reduced by changing the implementation platform rather than with a substantially simpler model.

Through verifying the functionality of the system via simulation and on the physical FPGA, we reduce the likelihood that discrepancies observed in later experiments were caused by functional or timing errors in the hardware implementation.

B. Software and Hardware Comparison

Once we had validated the hardware and simulation matched, we put around 874,700 samples of IMU data through both the simulated hardware model and software model from [7]. The mean absolute error (MAE), maximum absolute error (MaxAE), and standard deviation of the error were calculated for each of nine IMU features.

These metrics were also expressed relative to the feature's standard deviation. This was to create a reference scale so that the magnitude of the implementation error could be considered relative to the variation present in each signal. This software to hardware comparison evaluates whether we successfully preserved the behaviour of the software reference. This is particularly important since an objective of this work is to accelerate the CNN-based approach by Kayan et al. in [7] rather than develop a novel anomaly detection method. Substantial numerical differences between the two implementations would compromise this goal.

TABLE VI
Mean, Maximum, and Standard Deviation of Absolute Error

Feature	Mean	Max	Std	Mean %	Max %	Std %
AccX	0.004346	0.03988	0.004377	0.29	2.63	0.289
AccY	0.009046	0.05333	0.007934	0.29	1.73	0.257
AccZ	0.0006449	0.004768	0.0005311	0.27	2.03	0.226
GyroX	0.08691	0.734	0.08336	0.31	2.62	0.297
GyroY	0.03005	0.3078	0.02875	0.37	3.81	0.356
GyroZ	0.08912	0.5342	0.07226	0.34	2.02	0.274
MagX	0.02116	0.1587	0.01902	0.36	2.72	0.325
MagY	0.04867	0.3639	0.05412	0.25	1.89	0.281
MagZ	0.08019	0.3487	0.05754	0.25	1.11	0.183

As shown in Table VI, the mean absolute error across all nine features relative to their standard deviations is quite small, with these ranging from 0.25–0.37%. These indicate that, on average, the numerical discrepancy between the hardware and software models is less than a third of a percent of the observed variation in a feature. This provides strong evidence that the hardware implementation closely reproduces the numerical behaviour of the software model across the samples tested.

While the maximum absolute error ranges from 1.11–3.81%, it is considerably larger than the mean. As the standard deviation of error for all nine features is quite small, this suggests the maximum errors occur in a small number of outliers rather than being typical hardware behaviour.

This close agreement is particularly significant when considered alongside the literature reviewed in Table I. Kayan et al. demonstrated the feasibility of IMU-based anomaly detection on an edge device but reported an inference latency of 115–120 ms. This result indicates that the same software model can be implemented and accelerated in FPGA hardware without introducing substantial numerical deviation.

Furthermore, this close alignment also demonstrates that using round to zero has a negligible impact on the model's numerical behaviour, making it a suitable choice to reduce

TABLE VII
ACCUMULATED RMSE ($W = 200$) USING MODEL FROM [7]

Dataset	Mean	Std	Min	Max
Normal Data	488.68	27.88	370.38	583.74
Hitting Arm	833.83	316.83	319.65	2021.11
Hitting Platform	585.24	36.42	459.62	710.45
Earthquake	656.27	40.12	512.09	785.19
Extra Weight	583.18	37.43	420.96	697.02
Magnet	29692.82	11477.04	3758.06	69839.51

the resource usage of the design while maintaining numerical accuracy.

C. Model Performance

1) Reproducing the Results: While we attempted to replicate the experiments conducted by Kayan et al. in [7] to achieve the same metrics for accuracy, precision, F1, and FPR, we were unable to do so. Rather than attributing this discrepancy immediately to the hardware implementation, we first isolated the software reference model to determine whether the same behaviour could be reproduced without the numerical differences introduced by the hardware model being present in the processing pipeline.

We isolated the hardware model by using the software model in [7] for our predictions, then we ran the same experiment and obtained the distribution of the RMSE for each of the datasets in [7], shown in Table VII.

The RMSE values were computed by getting a prediction from the model in [7] and comparing it to the ground truth. A window of size 200 was used for Table VII. The same pre-processing steps and files were used as in [7].

From Table VII we see that the threshold of approximately 465 stated in the source code of [7] is insufficient to properly distinguish between when the robot is operating normally and when it is in an anomalous state with a false positive rate (FPR) of zero. We see that the minimum accumulated RMSE of the normal data was around 370, and the mean around 489. Consequently, using such a low threshold would classify many of the normal windows as anomalous. As a result of this, we have been unable to properly reproduce the published results.

The above results suggest there may be a mismatch between the model provided in the repository and the model which was used for the results in [7]. This is because the discrepancy exists when using their own model, with the hardware absent from the pipeline. While a lot of care has been taken to ensure that the same pre-processing and datasets had been used, this discrepancy may be due to a bug in the software used when attempting to replicate the experiment.

At this point in time, we are unable to determine the exact cause of this discrepancy. In the interest of time, we continued to proceed using the available datasets and model to characterise the behaviour obtainable from the available implementation.

2) Software Results: Using the software model provided in the repository of [7], we performed our own exhaustive threshold search to reproduce their results. Our results are shown in Table VIII.

TABLE VIII
OPTIMAL METRICS OBTAINED USING MODEL FROM [7]

Window	Threshold	F1	Precision	Recall	Accuracy	FPR
45	141.14	0.638	0.657	0.619	0.812	0.1181
50	162.78	0.651	0.756	0.572	0.836	0.0673
100	297.12	0.710	0.759	0.667	0.855	0.0771
150	421.38	0.809	0.792	0.827	0.896	0.0788
200	547.80	**0.952**	**0.974**	**0.931**	**0.975**	**0.0088**
265	734.49	0.936	0.964	0.910	0.967	0.0122

TABLE IX
METRICS USING HARDWARE THRESHOLDS

Window	Threshold	F1	Precision	Recall	Accuracy	FPR
200	548.00	**0.951**	**0.978**	**0.925**	**0.975**	**0.0076**
265	732.81	0.934	0.963	0.906	0.966	0.0124

Like Table XI reported in [7], our metrics in Table VIII generally increase with increasing window size. However, this is not always the case: increasing the window from 200 to 265 lowers the F1 score from 0.952 to 0.936, with recall falling from 0.931 to 0.910.

This demonstrates that simply increasing the window size does not guarantee an improvement in model performance. An anomaly may be a short event, so a larger window dilutes it with normal data, making the accumulated RMSE less sensitive to brief anomalies such as hitting the arm or platform.

We also note that the 1000-point threshold grid used in [7] underestimates the achievable F1 score here, as the magnet data stretches the range of the accumulated RMSE and makes the grid too coarse: at $W = 200$ it gives 0.891, versus 0.952 for the exhaustive search.

3) Hardware Results: Since the optimal values were from the windows of size 200 and size 265, we ran the hardware model with these RMSE window sizes and the same exhaustive threshold search since they used different precisions, with Table VIII being single-precision (float32) and Table IX being half-precision (float16).

We see that the differences between the metrics of the software and hardware models are quite small, with these being at most 0.006. The largest difference is in recall, which is slightly lower in hardware at both window sizes. This is consistent with a small systematic bias: since the hardware rounds toward zero, its accumulated RMSE is on average around 0.2% lower than the software reference, so a few borderline anomalous windows fall just below the threshold. In addition to the earlier software to hardware numerical comparison, this provides further evidence that the hardware implementation does not introduce a substantial degradation in model performance.

Therefore, these results indicate that a hardware implementation can closely replicate both the numerical output and performance of a software implementation. This is shown by the very similar metrics produced by both models with the same sized RMSE windows and thresholds.

Furthermore, these experiments suggest that the inability to reproduce the results in [7] is unlikely to be due to the hardware implementation as the discrepancy was observed when

using the provided software model directly. The more likely explanation is that there are differences between the publicly available implementation and the experimental configuration used to produce the original results, although available evidence is insufficient to identify the precise cause.

Consequently, these results indicate that FPGA can reproduce the available software model's behaviour while significantly reducing the inference latency. This directly addresses the gap identified in the literature review, where existing edge implementations such as [7] achieve low power but at high latencies, while GPU-based approaches achieve lower inference latency at a significantly larger power consumption. The proposed FPGA implementation combines a measured inference latency of 120 ns with an estimated power consumption of approximately 1.684 W during inference and 0.227 W while idle. This power consumption is significantly lower than the tens to hundreds of watts range of the GPU implementations in Table I as well as the power of some of the edge devices such as Nvidia Jetson and Raspberry Pi 4.

While the estimated inference power consumption of the proposed design is significantly larger than the microcontroller designs, it is a trade-off which is unavoidable since we need larger computing power to achieve lower latency.

V. Conclusion

Despite the inability to reproduce the results from [7], the proposed design in this paper demonstrates that FPGA can be used to design anomaly detection systems which are both low latency and low power, and applies FPGA inference to externally sensed motion monitoring of robotic arms, addressing the gaps identified in the literature.

While we demonstrate the principle of eliminating all software and OS overhead from the critical path in the context of CNN-based anomaly detection, these ideas are transferable to other contexts which would yield similar gains in performance. These include situations where the latency is dominated by peripheral or bus transfers rather than compute as would be the case when small models are run on TPUs or GPUs, and when the model is small enough to fit in reconfigurable fabric.

However, this work has several limitations. While the hardware was verified against both simulation and the software model of [7], we were unable to reproduce the detection metrics published in [7]. Our results indicate that this is unlikely to be caused by the hardware implementation, although the exact cause is still unknown. The power figures are estimates from Vivado rather than measurements, the on-board test used a 600-row test vector while the full dataset was evaluated in simulation, and the design uses 87% of the available LUTs, leaving little room for larger models. Future work would therefore be to resolve the replication discrepancy, measure the power consumption on the board, and investigate fixed-point arithmetic to reduce resource usage.

VI. Acknowledgement of AI

AI was used in code reviews, writing boilerplate, drawing diagrams, and proofreading.

References

[1] Int. Federation of Robotics, "Global robot demand in factories doubles over 10 years," IFR, Frankfurt, Germany. [Online]. Available: https://ifr.org/ifr-press-releases/news/global-robot-demand-in-factories-doubles-over-10-years

[2] Universal Robots, "Real-time data exchange (RTDE)." [Online]. Available: https://www.universal-robots.com/developer/communication-protocol/rtde/

[3] RCC, "Industrial robot failure: Causes and prevention." [Online]. Available: https://www.rcc.fr/en/industrial-robot-failure-causes-prevention/

[4] Adams Corp, "Why industrial robots fail and how to prevent it." [Online]. Available: https://adamscorp.com/our-blog/why-industrial-robots-fail-and-how-to-prevent-it

[5] H. Kayan, R. Heartfield, O. Rana, P. Burnap, and C. Perera, "CASPER: Context-aware IoT anomaly detection system for industrial robotic arms," *ACM Trans. Internet Things*, vol. 5, no. 3, Art. 18, 2024, doi: 10.1145/3670414.

[6] Alias Robotics, "Week of Universal Robots bugs exposing insecurity." [Online]. Available: https://aliasrobotics.com/news/week-of-universal-robots-bugs-exposing-insecurity.php

[7] H. Kayan, R. Heartfield, O. Rana, P. Burnap, and C. Perera, "Real-time anomaly detection for industrial robotic arms using edge computing," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 29696–29712, 2025, doi: 10.1109/JIOT.2025.3569780.

[8] Engineering.com, "How important is high speed in robotic assembly?" [Online]. Available: https://www.engineering.com/how-important-is-high-speed-in-robotic-assembly/

[9] A. Koksal, "AI hardware accelerators explained: NPUs, TPUs, and GPUs," HP Tech Takes, Feb. 2026. [Online]. Available: https://www.hp.com/us-en/tech-takes/components/explainer/ai-hardware-accelerators-npu-tpu-gpu-guide.html

[10] S. Hosseininoorbin, S. Layeghy, M. Sarhan, R. Jurdak, and M. Portmann, "Exploring Edge TPU for network intrusion detection in IoT," arXiv:2103.16295.

[11] Arm, "Arm Ethos-U Vela compiler." [Online]. Available: https://arm-software.github.io/CMSIS-Ethos-U/main/vela/index.html

[12] M. Antonini, M. Pincheira, M. Vecchio, and F. Antonelli, "An adaptable and unsupervised TinyML anomaly detection system for extreme industrial environments," *Sensors*, vol. 23, no. 4, Art. 2344, 2023, doi: 10.3390/s23042344.

[13] AIR Journal, "Edge AI for real-time anomaly detection in industrial IoT: A hybrid framework." [Online]. Available: https://airjournal.org/article/edge-ai-for-real-time-anomaly-detection-in-industrial-iot-a-hybrid-framework-w6jq7/

[14] T. Kang, B.-J. You, J. Park, and Y. Lee, "A real-time anomaly detection method for robots based on a flexible and sparse latent space," arXiv:2504.11170.

[15] J. Li, X. Liu, X. Wu, D. Wang, K. Xu, and Y. Li, "A time and frequency convolutional autoencoder for anomaly detection in industrial robots based on IMU error calibration," *Eng. Appl. Artif. Intell.* [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197625022778

[16] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, "Real-time anomaly detection and reactive planning with large language models," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[17] Y. Wang, K. Pan, T. Song, L. Shi, and Z. Sui, "Hybrid wavelet transform and deep learning method for fault diagnosis of transformer vibration signals." [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1110016826001948

[18] H. Xie *et al.*, "TurboVLA: Real-time vision-language-action model at 32 Hz on an RTX 4090 with $<$1 GB VRAM," arXiv:2607.27205.

[19] J. Li, M. Cheng, Y. Wei, and Z. Dai, "FPGA implementation of edge-side motor fault diagnosis using a Kalman filter-based empirical mode decomposition algorithm." [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0967066125000759

[20] C.-C. Chung, Y.-P. Liang, and H.-J. Jiang, "CNN hardware accelerator for real-time bearing fault diagnosis," *Sensors*, vol. 23, no. 13, Art. 5897, 2023, doi: 10.3390/s23135897.

[21] S. T. Roche *et al.*, "Nanosecond anomaly detection with decision trees and real-time application to exotic Higgs decays," *Nat. Commun.*, vol. 15, Art. 3527, 2024, doi: 10.1038/s41467-024-47704-8.

All online sources accessed Sep.–Oct. 2026.